🚗 Parking Management System

A backend application built using **Spring Boot, Spring Data JPA, and MySQL** to manage parking-related operations through REST APIs.

The system provides a structured backend for managing parking slots, vehicles, parking entries, and parking records.

📌 Project Overview

The **Parking Management System** is a Spring Boot REST API application designed to simplify parking management.

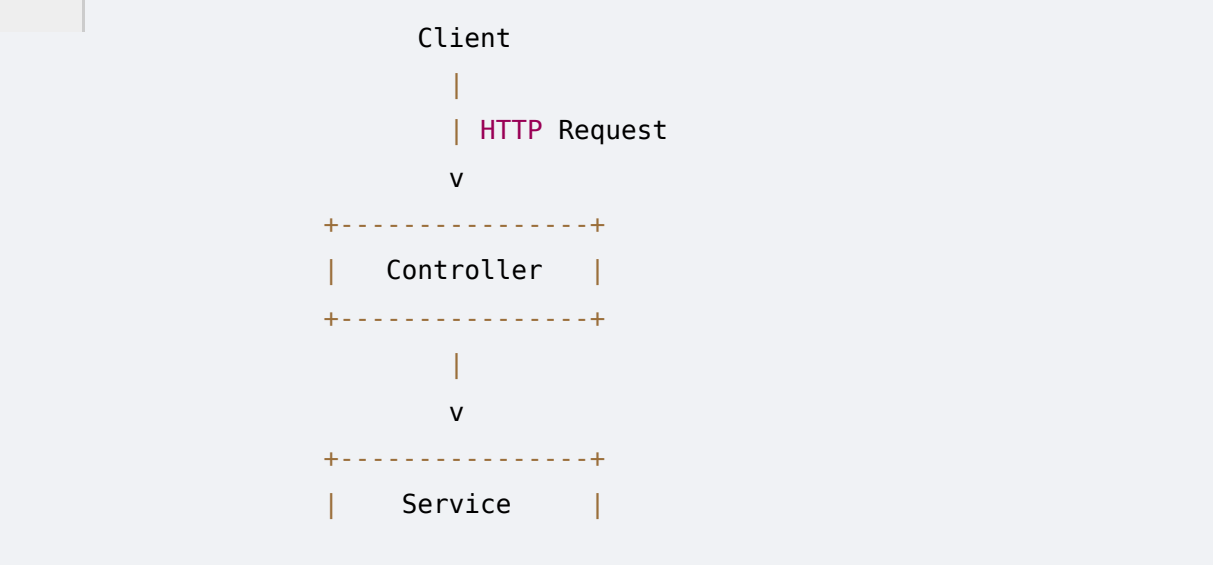
The application uses:

- **Spring Boot** for backend development
- **Spring Web MVC** for REST APIs
- **Spring Data JPA** for database operations
- **Hibernate/JPA** for ORM
- **MySQL** for persistent data storage
- **Maven** for dependency management and build
- **Java 21** as the programming language

The current project configuration is based on Spring Boot `4.1.0`, Java `21`, Spring Data JPA, Spring Web MVC, and MySQL.

🏗️ Architecture

The application follows a layered Spring Boot architecture:



```
                +----------------+
                        |
                        v
                +----------------+
                |   Repository   |
                +----------------+
```

Request Flow

```
Client
   ↓
Controller
   ↓
Service
   ↓
Repository
   ↓
JPA / Hibernate
   ↓
MySQL
   ↓
Response
   ↓
Client
```

🛠️ Tech Stack

Technology	Purpose
Java 21	Programming Language
Spring Boot 4.1.0	Backend Framework
Spring Web MVC	REST API Development
Spring Data JPA	Database Access
Hibernate	ORM
MySQL	Database

Maven	Build & Dependency Management
JUnit / Spring Boot Test	Testing

The dependencies are defined in the project's `pom.xml`.

📂 Project Structure

A recommended structure for the Parking Management System is:

```
parking-management-system
│
├── src
│   ├── main
│   │   ├── java
│   │   │   └── com.example.parkingmanagementsystem
│   │   │       │
│   │   │       ├── controller
│   │   │       │
│   │   │       ├── service
│   │   │       │
│   │   │       ├── repository
│   │   │       │
│   │   │       ├── entity
│   │   │       │
│   │   │       └── ParkingManagementSystemApplication.java
│   │   │
│   │   └── resources
```

🚗 Core Modules

The application can be organized around the following modules.

1. Vehicle Management

Responsible for managing vehicles entering and leaving the parking facility.

Typical operations:

```
Add Vehicle
Get Vehicle
Update Vehicle
Delete Vehicle
```

---

2. Parking Slot Management

Responsible for managing available and occupied parking slots.

Typical operations:

```
Create Parking Slot
View Parking Slots
Check Slot Availability
Update Slot Status
```

Example slot states:

```
AVAILABLE
OCCUPIED
```

---

3. Parking Management

Responsible for maintaining parking records.

A parking record can contain information such as:

```
Vehicle
Parking Slot
Entry Time
Exit Time
Parking Status
```

🌐 REST API

The backend exposes REST APIs using Spring Web MVC.

Typical API structure:

```
POST    /api/vehicles
GET     /api/vehicles
GET     /api/vehicles/{id}
PUT     /api/vehicles/{id}
DELETE  /api/vehicles/{id}
```

Parking slot APIs:

```
POST    /api/parking-slots
GET     /api/parking-slots
```

```
GET     /api/parking-slots/{id}
PUT     /api/parking-slots/{id}
DELETE  /api/parking-slots/{id}
```

Parking APIs:

```
POST    /api/parking
GET     /api/parking
GET     /api/parking/{id}
PUT     /api/parking/{id}
DELETE  /api/parking/{id}
```

> These endpoints are the intended API structure for the README; the uploaded files do not currently provide controller implementations to verify exact endpoint names.

🗄️ Database

The application uses **MySQL** as its database.

Spring Data JPA is used to communicate with the database.

```
Spring Boot
     ↓
Spring Data JPA
     ↓
Hibernate
     ↓
JDBC
     ↓
MySQL
```

The project includes the MySQL Connector/J runtime dependency in `pom.xml`.

⚙️ Configuration

Database configuration can be provided in:

```
src/main/resources/application.properties
```

Example:

```
spring.datasource.url=jdbc:mysql://localhost:3306/parking_management
spring.datasource.username=root
spring.datasource.password=your_password

spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
```

🔄 JPA Layer

The application uses Spring Data JPA for database operations.

Typical architecture:

```
Entity
  ↓
Repository
  ↓
JPA / Hibernate
  ↓
MySQL
```

🧪 Testing

The project includes Spring Boot testing dependencies.

Testing dependencies include:

```
spring-boot-starter-test
spring-boot-starter-data-jpa-test
spring-boot-starter-webmvc-test
```

These are configured with test scope in `pom.xml`.

Tests can be created under:

```
src/test/java
```

▶️ Running the Application

1. Clone the repository

```
git clone <repository-url>
```

2. Open the project

Open the project in:

```
IntelliJ IDEA
```

or another Java IDE.

3. Configure MySQL

Create the database:

```
CREATE DATABASE parking_management;
```

Update:

```
application.properties
```

with your MySQL username and password.

4. Build the project

Using Maven:

```
./mvnw clean install
```

Windows:

```
mvnw.cmd clean install
```

5. Run the application

```
./mvnw spring-boot:run
```

Windows:

```
mvnw.cmd spring-boot:run
```

🔍 API Testing

The APIs can be tested using:

- Postman

📊 Working Flow

A typical parking flow:

```
Vehicle arrives
      ↓
Vehicle details recorded
      ↓
Available parking slot identified
      ↓
Parking slot assigned
      ↓
Vehicle enters parking
      ↓
Entry time recorded
      ↓
Vehicle exits
      ↓
Exit time recorded
      ↓
Parking duration calculated
      .
```

🚀 Future Enhancements

The project can be extended with:

- JWT Authentication
- Role-based authorization
- Admin dashboard
- Parking slot search
- Automatic fee calculation
- Vehicle entry/exit tracking
- Payment integration
- Global exception handling
- DTO layer
- Validation
- Swagger/OpenAPI
- Docker
- Docker Compose
- CI/CD using GitHub Actions
- Microservices architecture
- Eureka Service Discovery

- API Gateway
- OpenFeign communication
- Cloud deployment

📚 Reference Documentation

The project was generated/configured around the following Spring documentation:

- Apache Maven Documentation
- Spring Boot Maven Plugin
- Spring Web
- Spring Data JPA
- Building RESTful Web Services
- Accessing Data with JPA
- Accessing Data with MySQL

These references are also listed in the project's `HELP.md`.

👨‍💻 Project Summary

Parking Management System is a Spring Boot backend application that provides REST APIs for managing parking operations.

The project demonstrates:

```
Java 21
   ↓
Spring Boot
   ↓
Spring MVC
   ↓
REST APIs
   ↓
Service Layer
   ↓
Spring Data JPA
   ↓
Hibernate
   ↓
MySQL
```